Open in Colab

Model parameters vs hyperparameters

1. Model Parameters

- **Definition**: Parameters are the values that the model **learns from the training data** during the training process.
- **Key Characteristics**:
  - They are **internal to the model** and adjusted automatically during training.
  - They directly influence the predictions or output of the model.
  - Their values depend on the training data and optimization process.
- **Examples**:
  - In a **Linear Regression** model:
    - The coefficients (w) and the intercept (b) are parameters.
  - In a **Neural Network**:
    - The weights and biases in each layer are parameters.
  - In **K-Means** clustering:
    - The **centroids** of the clusters are parameters.

2. Model Hyperparameters

- **Definition**: Hyperparameters are **external settings** for the model that are **not learned from the data** but are set **before training** begins.
- **Key Characteristics**:
  - They are used to control the learning process or the structure of the model.
  - They are tuned **manually** (or via search methods like grid search) to optimize the model's performance.
  - Hyperparameters must be chosen carefully because they can significantly impact the model's ability to generalize.
- **Examples**:

- In **K-Means** clustering:
  - The number of clusters (k) is a hyperparameter.
- In **DBSCAN**:
  - Epsilon (ϵ) and `min_samples` are hyperparameters.
- In **Neural Networks**:
  - The number of layers, number of neurons, learning rate, and batch size are hyperparameters.
- In **Decision Trees**:
  - Maximum tree depth (`max_depth`), minimum samples per split (`min_samples_split`), etc., are hyperparameters.

Summary: Key Differences

Feature	Model Parameters	Hyperparameters
Definition	Learned from the data during training	Set manually before training
Adjustable?	Adjusted by the optimization algorithm	Set by the user or via tuning techniques
Examples	Weights in Neural Networks	Learning rate, number of layers
Model Training	Updated during training	Remain fixed during training

---

Hyperparameters (Tunning/Selection) For DBSCAN Algorithm

Determining the **hyperparameters** for the **DBSCAN** algorithm, namely **epsilon (ϵ)** and **minimum points (`min_samples`)**, can be challenging. However, there are systematic methods to estimate these values effectively before fine-tuning through trial and error.

1. Determining Epsilon (ϵ)

The **epsilon** parameter defines the neighborhood radius around each point. Points within this radius are considered neighbors.

Techniques to Estimate Epsilon

1. **k-Distance Plot (k-NN Distance Plot)**:
   - Compute the distances to the **k-th nearest neighbor** for all points (where k = min_samples).
   - Sort the distances in ascending order and plot them.

- Look for the "**elbow point**" where the slope changes significantly. This is a good candidate for ϵ.
- The elbow point in the plot suggests the optimal ϵ.
- If the plot lacks a clear elbow, test a range of values manually.

2. **Domain Knowledge**:
   - If you know the scale or spread of the data, you can estimate ϵ based on typical distances between points.
   - Example: In geographic data, ϵ could correspond to a known radius (e.g., kilometers).
3. **Distance Metrics**:
   - Inspect the pairwise distances between points to understand the average neighborhood radius.

2. Determining `min_samples`

The `min_samples` parameter defines the minimum number of points (including the point itself) that must exist in a neighborhood of radius ϵ for the point to be considered a **core point**.

Techniques to Estimate `min_samples`

1. **Rule of Thumb**:
   - Start with:
   $$\text{min_samples} = \text{number of features} + 1$$
   - For example:
     - 2D data $\rightarrow$ `min_samples = 3`.
     - 3D data $\rightarrow$ `min_samples = 4`.
2. **Domain Knowledge**:
   - If you know the minimum density required for clusters, you can set `min_samples` accordingly.
   - Example: In customer segmentation, you might set `min_samples` to 5 or 10 to define a minimum cluster size.
3. **Experimentation**:
   - Run DBSCAN with a range of `min_samples` values (e.g., 3–10) and evaluate the resulting clusters visually or using metrics like the Silhouette Score.

3. Combining ϵ and `min_samples`

1. **Iterative Refinement**:

- Start with the **k-distance plot** to find an initial estimate for ϵ.
- Use `min_samples = 4` or a value based on the number of features.
- Run DBSCAN and evaluate the results.
- Adjust ϵ and `min_samples` iteratively to fine-tune the clustering.

2. **Visualization**:
   - Visualize the clusters using scatter plots to ensure they align with expectations.
   - Points labeled as **-1 (noise)** should make sense intuitively.

4. Validation Metrics

After selecting ϵ and `min_samples`, validate the clustering using metrics:

1. **Silhouette Score**: Measures how well each point fits into its cluster.

```
from sklearn.metrics import silhouette_score
silhouette_score(X, dbscan_labels)
```

2. **Number of Clusters**: Check if the number of clusters is reasonable for your data.
3. **Visual Inspection**: Plot the clusters and noise points to ensure they align with the data's structure.

Summary

1. Use the **k-distance plot** to estimate ϵ.
2. Start with `min_samples = 4` (rule of thumb) or based on domain knowledge.
3. Fine-tune ϵ and `min_samples` iteratively, evaluating results visually and using metrics like the Silhouette Score.

Practical Example using Python

In this example, we will use the Mall Customers Dataset which we used in KMEANs clustering section and we will try to determine the initial hyperparameters using the methods discussed above.

Get the Data

We will import mall customers data and extract two features only and use them to create our clusters.
We will choose 2 features to make plotting easier for learning purposes.

```
# Import necessary libraries
import pandas as pd
import numpy as np
```

```
# Get the data
df = pd.read_csv('https://raw.githubusercontent.com/msfasha/307304-Data-Mining/refs/heads/ma
features = df[["Annual Income (k$)", "Spending Score (1-100)"]]
```

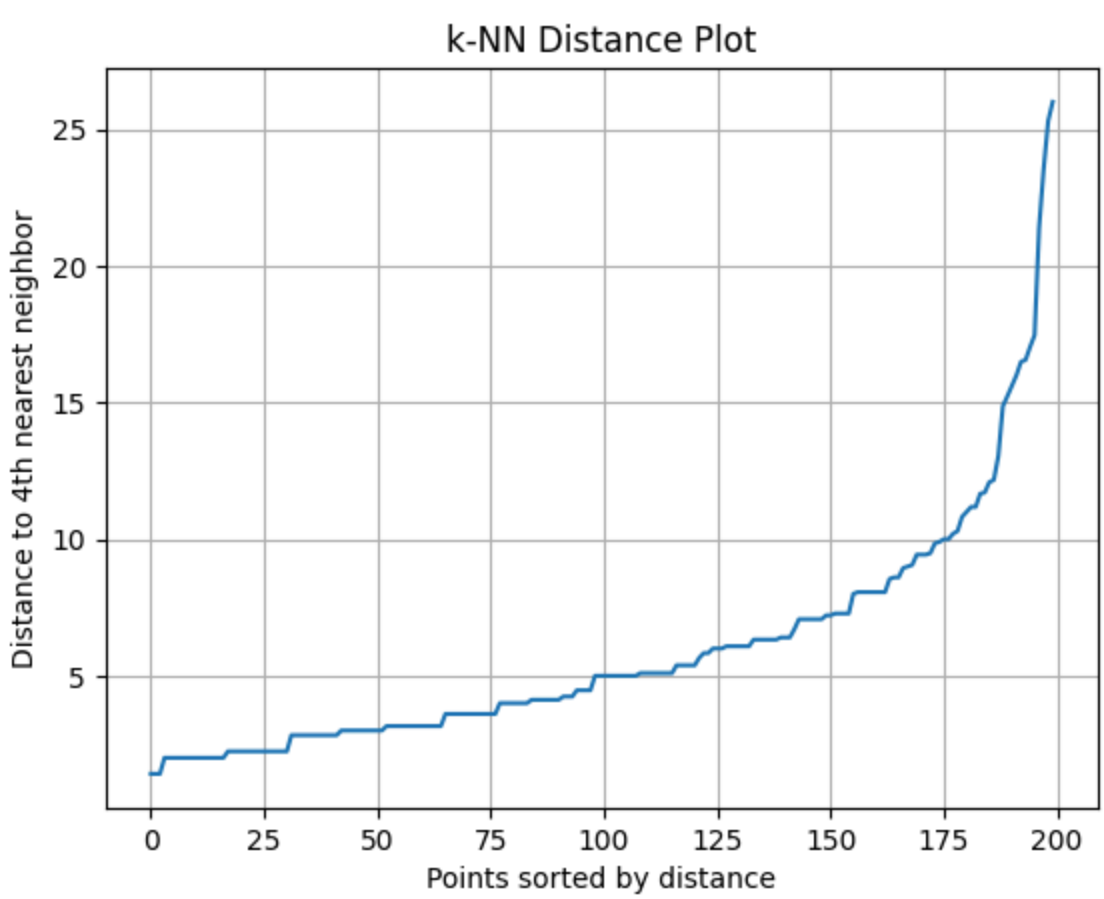
Examine Dataset using the Elbow Method

```
# Import necessary libraries

import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors

# Compute distances to the k-th nearest neighbor
neighbors = NearestNeighbors(n_neighbors=4)  # Use min_samples = 4
neighbors_fit = neighbors.fit(features)
distances, _ = neighbors_fit.kneighbors(features)

# Sort the distances to the k-th nearest neighbor
distances = np.sort(distances[:, 3])  # 3rd neighbor (0-based index)
plt.plot(distances)
plt.title("k-NN Distance Plot")
plt.xlabel("Points sorted by distance")
plt.ylabel("Distance to 4th nearest neighbor")
plt.grid()
plt.show()
```



Run DBSCAN using Distance Value based on the previous Elbow Plot

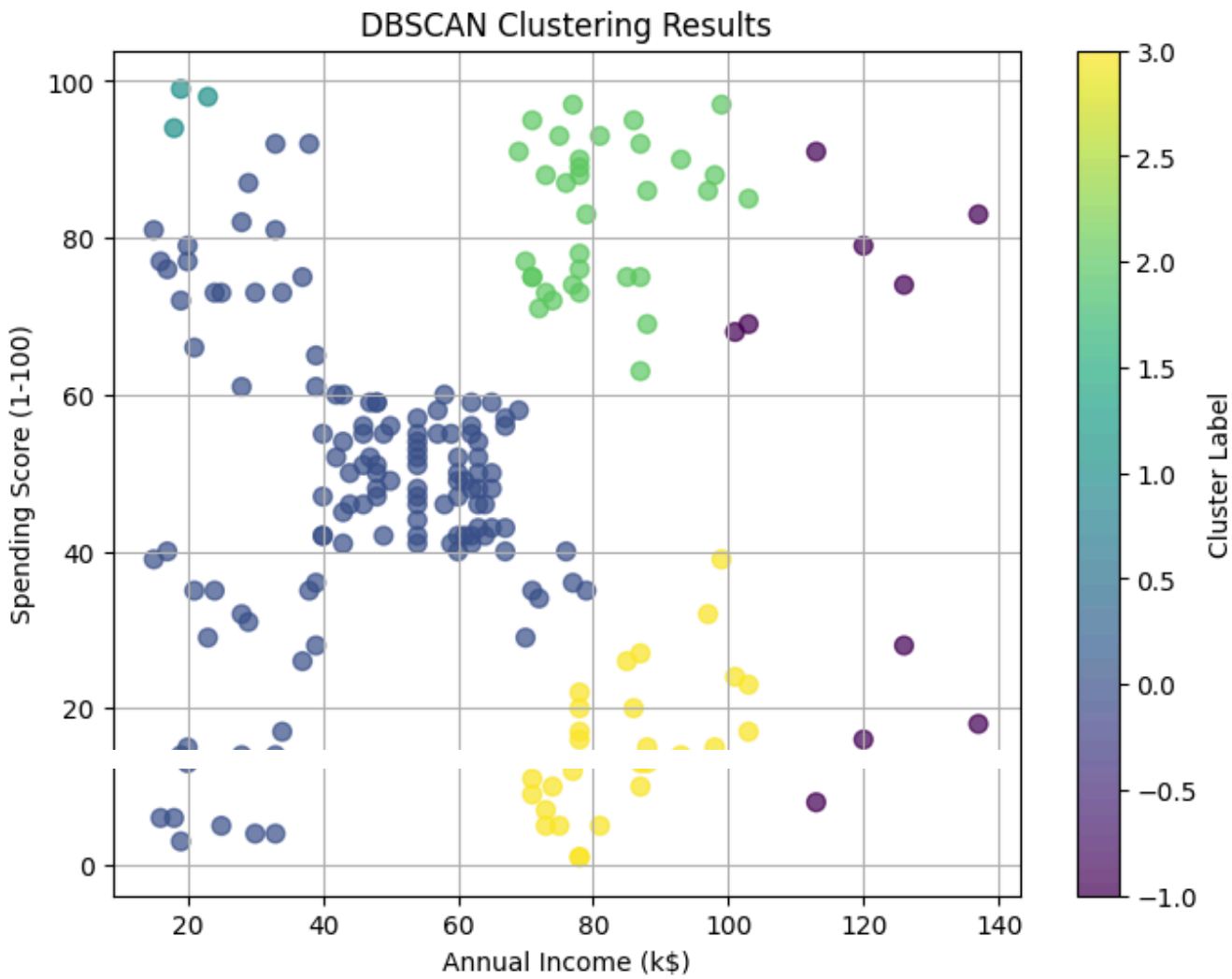
```
# Import necessary libraries
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# Run DBSCAN
epsilon = 10  # Use the elbow distance (you can tune this)
min_samples = 3  # Based on your earlier k-NN distance plot
dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
labels = dbscan.fit_predict(features)

# Add cluster labels to the original dataframe
df['Cluster'] = labels

# Visualize the clustering results
plt.figure(figsize=(8, 6))
plt.scatter(df["Annual Income (k$)"], df["Spending Score (1-100)"], c=labels, cmap='viridis'
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("DBSCAN Clustering Results")
plt.colorbar(label="Cluster Label")
plt.grid()
plt.show()
```

```
# Print summary
num_clusters = len(set(labels)) - (1 if -1 in labels else 0)  # Exclude noise label (-1)
print(f"Number of clusters (excluding noise): {num_clusters}")
print(f"Number of noise points: {list(labels).count(-1)}")
```




```
Number of clusters (excluding noise): 4
Number of noise points: 10
```

Validate Clustering with Silhouette Score

The **silhouette score** is a metric used to evaluate the quality of clustering. It combines both cohesion (how close the points within a cluster are) and separation (how distinct a cluster is from others). The silhouette score ranges from -1 to +1:

- **+1** indicates that the points are well-clustered, meaning they are close to other points in the same cluster and far from points in other clusters.
- **0** indicates that the points are on or near the boundary between two clusters.

- **-1** indicates that the points are incorrectly clustered, as they are closer to points in a different cluster than to points in their own cluster.

It is commonly used in clustering algorithms like K-means or hierarchical clustering to assess how well the clustering algorithm has performed.

```
from sklearn.metrics import silhouette_score

# Compute Silhouette Score
if len(set(labels)) > 1:  # Avoid silhouette score error for single cluster
    score = silhouette_score(features, labels)
    print(f"Silhouette Score: {score:.2f}")
else:
    print("Silhouette Score cannot be calculated for a single cluster.")
```

```
Silhouette Score: 0.36
```